

5G Core API Security Project

Demo Guide

A structured handout for presenting the dashboard, scanner CLI, implementation logic, and practical security work.

Project Focus	5G Core Service-Based Interface security
Main Components	NRF, UDM, NEF-inspired API flows
Interfaces Used	Dashboard + Scanner CLI
Goal of Demo	Show vulnerable vs secure behavior with measurable evidence

1. Demo Preparation

- Open a terminal in **D:\5G-Project**.
- Start the web application with **python -m uvicorn app.main:app --reload**.
- Open the dashboard at **http://127.0.0.1:8000**.
- Keep a second terminal ready for the CLI demonstration.

Suggested opening line:

“My project is a 5G Core API security testing platform. It demonstrates how vulnerable and secure microservice APIs behave under injection, authentication, and authorization attacks.”

2. Dashboard Demo Flow

1. Introduce the attack surface: Explain that the platform focuses on Service-Based Interfaces in the 5G Core and uses components inspired by NRF, UDM, and NEF.
2. Show the Attack Surface area: Present the simulated assets, API exposure, and security indicators.
3. Run the first attack: Click **Load Command Sample**, then **Run Injection Test**. Explain that the vulnerable path shows unsafe command construction while the secure path blocks malicious input.
4. Run a second attack: Use **Load SUPI Injection** or **Load JSON Injection**, then run the test. Explain how crafted subscriber identity or JSON input affects weak API logic.
5. Launch a structured scan: Keep the default selected modules and click **Launch Structured Scan**. Explain that this runs multiple checks and stores formal findings.
6. Open Recent Runs: Click a recent run and show that the platform stores scan evidence for later review.
7. Open Findings: Use **Refresh Catalog** and **Refresh Fuzz** to show attack categories and OpenAPI-guided fuzz inputs.
8. Open Threats: Use **Show Lab** and **Verify JWT** to show the lab blueprint and token-verification scenario.
9. Show support sections: Use **Show Playbook** and **Show Sources** to connect the work to standards, best practices, and research.

Useful sentence during the dashboard demo:

“The dashboard gives me a visual way to explain the story: attack surface, attack execution, mitigation, and evidence.”

3. CLI Demo Flow

Run the following commands in a second terminal:

Command	Purpose
<code>python main.py --mode scan</code>	Run the scanner in scan mode
<code>python main.py --mode scan --nf UDM</code>	Focus the scan on subscriber-related checks
<code>python main.py --mode attack --module udm</code>	Show a targeted offensive module
<code>python scanner_cli.py profiles list</code>	List scanner profiles
<code>python scanner_cli.py run --profile mock-local-lab</code>	Run the structured engine without active traffic

Suggested explanation: “The CLI proves that the platform is not only a dashboard. It also supports structured, modular security testing like a real scanner.”

4. Short Conclusion for the Demo

“In conclusion, this project combines 5G Core concepts, API security testing, a dashboard, a scanner CLI, and measurable evidence into one coherent platform.”

5. Q&A; for the Demo, Implementation, and Practical Work

Q: Why did you focus on the 5G Core instead of general web security?

A: Because modern 5G Core networks are API-driven. Network Functions communicate like microservices, so API security becomes directly relevant to telecom security.

Q: What is the main idea of your project?

A: It is a 5G Core API security testing platform that demonstrates vulnerable versus secure behavior under injection, authentication, and authorization attacks.

Q: Why are NRF, UDM, and NEF important in your project?

A: NRF represents service registration and discovery, UDM represents subscriber data, and NEF represents exposed service functions such as diagnostics. Together they cover realistic API security cases.

Q: What is the difference between the vulnerable path and the secure path?

A: The vulnerable path simulates unsafe behavior such as weak input handling or weak trust. The secure path applies validation, parameterized logic, and authorization checks to block the same payload.

Q: Why did you use OpenAPI in the project?

A: OpenAPI helps describe the attack surface in a structured way. It lets the platform discover endpoints, parameters, and generate more realistic fuzzing or test cases.

Q: What is BOLA in your project?

A: BOLA stands for Broken Object Level Authorization. It means changing an object identifier, such as a subscriber ID, to access data that should not be available to the caller.

Q: Why is JWT verification important here?

A: Because Network Functions may rely on tokens to trust each other. If claims are modified and not properly verified, a malicious service could gain unauthorized privileges.

Q: Why did you build both a dashboard and a CLI?

A: The dashboard is ideal for live explanation and visual evidence, while the CLI proves the platform also works as a modular scanner and not just as a user interface.

Q: What practical work did you actually implement yourself?

A: I implemented the FastAPI backend, the interactive dashboard behavior, the scanner routes, the modular CLI workflow, the evidence storage, and the attack simulation logic for multiple scenarios.

Q: How do you prove that the project really works?

A: I prove it by running live attack scenarios, showing vulnerable and secure outputs side by side, storing findings and logs, and demonstrating both dashboard and CLI workflows.

Q: What are the main limitations of the project?

A: It is a contained academic simulation, so it does not fully replace testing on a real 5G Core deployment. Its strength is safe demonstration, clear attack logic, and measurable evidence.

Q: What would you improve in the future?

A: I would integrate more directly with real free5GC or Open5GS labs, add stronger service-to-service trust such as OAuth2 or mTLS, and expand reporting and fuzzing coverage.

Best final summary answer:

"My project is a practical platform for understanding and demonstrating how API vulnerabilities can impact 5G Core microservices, and how secure controls can prevent those attacks."